

IRIS: Learning-Driven Task-Specific Cinema Robot Arm for Visuomotor Motion Control

Qilong Cheng
Mechanical and Aerospace Eng. Department
New York University
qc1007@nyu.edu

Matthew Mackay
MIE Department
University of Toronto
mackay@mie.utoronto.ca

Ali Bereyhi
ECE Department
University of Toronto
ali.bereyhi@utoronto.ca

Abstract—Robotic camera systems enable dynamic and repeatable motion beyond human capabilities. Yet their adoption is restricted by high costs and operational complexity of industrial-grade hardware. We present *intelligent robotic imaging system* (IRIS), a task-specific 6-DOF manipulator, designed for autonomous learning-driven cinematic motion control. Our system leverages a vertically integrated stack that combines a lightweight, 3D-printed hardware design with a visuomotor imitation learning framework. By employing a goal-conditioned adaptation of Action Chunking with Transformers (ACT), IRIS learns to execute object-aware, perceptually smooth trajectories directly from human expert demonstrations, without the need for explicit geometric programming. The complete system costs under \$1,000 USD, supports a 1.5 kg payload, and achieves approximately 1 mm repeatability. Real-world experiments demonstrate accurate tracking and reliable autonomous execution that generalizes across diverse cinematic trajectories. Implementation details for IRIS are available at: <https://github.com/thejerrycheng/iris>.

Index Terms—robotic motion control, imitation learning, visuomotor control, full-stack robotic systems

I. INTRODUCTION

A *cinema robot* is a robotic motion-control system, typically a multi-degree-of-freedom (multi-DOF) arm or rig, designed to execute smooth, precise, and repeatable camera motion for shot execution and visual-effects continuity [1], [2]. Such systems are widely used in film, commercials and television production to capture dynamic, high-speed, or multi-pass shots that demand sub-millimeter repeatability and precise temporal alignment [1], [3]. State-of-the-art cinema robots achieve high speed, large payload capacity, and high repeatability, but typically rely on industrial-grade actuation and control architectures that incur substantial cost and operational complexity [1], [4], [5]. As a result, these platforms remain largely inaccessible to independent creators and research environments. In contrast, recent low-cost and open-source robotic manipulators demonstrate the feasibility of 3D-printed hardware for learning-based control, but often trade off workspace, speed, or structural rigidity, limiting their suitability for professional cinematic camera motion [6]–[8].

This study explores a task-specific alternative: *what if a robotic arm were designed exclusively for cinematic motion, and learned camera behavior directly from expert operators?*

¹This work was conducted while Qilong Cheng was at the University of Toronto.



Fig. 1. Tabletop deployment of the IRIS prototype performing a real-world demonstration. An end-effector-mounted camera captures a target object (cup) using visuomotor control.

Our goal is to lower the barrier to professional camera motion control, and enable broader access for creators and researchers alike. We present *Intelligent Robotic Imaging System*, **IRIS**, a compact, low-cost, 3D-printed 6-DOF cinema robot designed explicitly for dynamic camera motion rather than adapted from industrial manipulators. Figure 1 shows the hardware prototype of IRIS in a tabletop setup. The key contributions of this work are as follows.

- **Task-specific hardware design.** A low-cost 3D-printed 6-DOF manipulator is co-designed to meet the reachability, accuracy, and repeatability for cinema motion control.
- **Goal-conditioned ACT learning.** An imitation learning framework is developed, which extends Action Chunking with Transformers (ACT) with visual goal conditioning to generate obstacle-aware, goal-directed camera motion directly from observations.
- **End-to-end real-world validation.** A vertically integrated system is implemented and validated on physical hardware, showing accurate low-level control and autonomous high-level motion control.

To the best of our knowledge, IRIS is among the first systems to prioritize low-cost cinematic shot automation through joint hardware–control co-design driven by robot learning.

The remainder of the paper is organized as follows: we

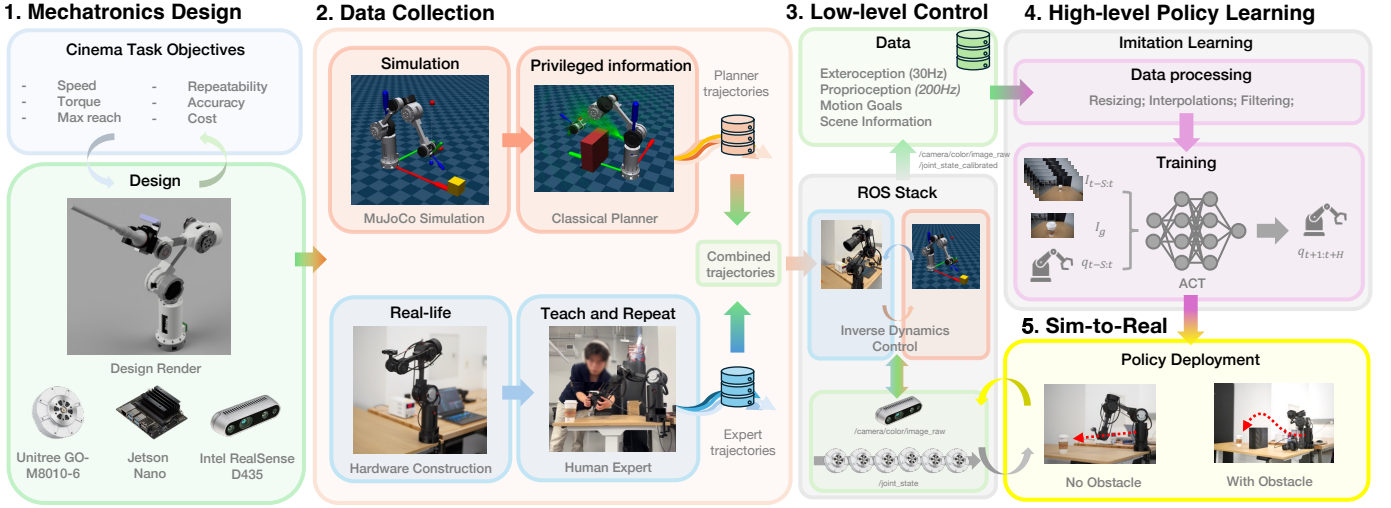


Fig. 2. Overview of the IRIS system pipeline. Cinema task objectives guide task-specific hardware design. Training data are collected exclusively from real-world human demonstrations, while classical planner trajectories generated in simulation are used for analysis and comparison. A ROS-based low-level control stack executes inverse-dynamics control, and a goal-conditioned imitation learning policy (ACT) is trained on human data and deployed on the physical robot, enabling smooth, obstacle-aware cinematic motion via sim-to-real transfer.

give an overview of the related work in Section II. Section III presents the mechatronics design of IRIS. Section IV describes the integration of the system, including simulation, ROS stack, and low-level control. The visuomotor imitation learning is presented in Section V. Section VI presents experimental evaluations on the physical system. Finally, Section VII concludes the paper and outlines future work. An overview of the complete system pipeline is shown in Figure 2.

II. RELATED WORK

In this section, we briefly discuss the cinema robot hardware and motion control methods relevant to our design goals.

A. State-of-the-art Cinema Robots

Robotic camera systems are widely used in professional film production due to their ability to execute precisely choreographed, highly repeatable motion required for visual-effects continuity and multi-pass composition [2], [4]. Commercial state-of-the-art cinema robots, e.g. the Bolt High-Speed Cinebot and Kira Cinema Robot [4], [5], achieve high speed, high payload capacity, and sub-millimeter repeatability, but rely on industrial-grade hardware that leads to high capital cost (often exceeding \$50,000 USD) and operational complexity. These systems are also typically operated by trained specialists and remain inaccessible to independent creators, small studios, and research environments.

Accessible and open-source 3D-printed manipulators such as Arctos, and Moveo [9], [10] demonstrate the feasibility of low-cost robotic arms using 3D-printing. But they often lack the workspace, speed, or sufficient payload required for cinematic camera motion. Those stepper-motor-based designs with high gear-ratio introduce cogging, backlash, and limited control bandwidth. To address these limitations, recent manipulators explore quasi-direct-drive (QDD) and remote-actuation architectures that reduce distal inertia and enable back-

drivable, low-impedance dynamics. Systems such as OpenArm and Berkeley Blue [7], [11] employ low-ratio brushless motors to achieve smooth, expressive motion suitable for dynamic tasks. IRIS adopts these design principles through a QDD, belt-driven architecture tailored for camera payloads, speed, achieving millimeter-level repeatability while maintaining the smoothness required for cinematic camera tracking. A detailed comparison of technical specifications between IRIS and existing state-of-the-art cinematic manipulators is provided in Table I.

B. Motion Control Methods

Most commercial cinema robots rely on classical motion generation techniques, including keyframe interpolation, time-parameterized splines, and inverse-kinematics-based trajectory replay [4]. To handle obstacle avoidance and workspace constraints, these systems often incorporate sampling-, graph-, or optimization-based planners such as A*, RRT*, CHOMP, and TrajOpt [12]–[15]. While those methods produce deterministic and geometrically feasible trajectories, they optimize hand-crafted objectives and fail to capture perceptual qualities central to cinematography, such as pacing, framing stability, and expressive smoothness.

Learning-based methods offer an alternative by learning planning objectives directly from data. Imitation learning (IL) enables end-to-end mapping from observations to actuator commands using expert demonstrations, and has been widely applied through behavior cloning (BC), dataset aggregation (DAgger), and generative adversarial formulations [16]–[18]. However, vanilla IL methods are sensitive to distribution shift and compounding errors. Recent transformer- and diffusion-based architectures address long-horizon temporal dependencies and compounding error by modeling temporally extended action sequences, enabling smooth, high-frequency control [19]–[21]. Inverse reinforcement learning (IRL) and guided cost learning extend this paradigm by inferring underlying cost functions

TABLE I
COMPARISON OF COMMERCIAL CINEMA ROBOTS AND 3D-PRINTED ROBOT ARMS FOR CINEMATIC APPLICATIONS.

Robot Arm	Reach (mm)	Speed (m/s)	Acceleration (m/s ²)	Repeatability (mm)	Payload (kg)	Ease of Use	Cost (USD)
Commercial Cinema Robots							
Bolt Cinebot [4]	2000	5.0	30	± 0.05	20	Medium	\$69k+
Colossus [27]	1100	6.8	40	± 0.05	10+	Medium	\$200k+
Kira [5]	1500	3.5	20	± 0.05	10	Low	\$50k+
E-Jib Mini [28]	1800	1.0	5	± 0.1	5	Low	\$9k
3D-Printed / Open-Source Arms							
Dexter [29]	700	0.5	2	± 0.025	3	Low	~\$2k
Arctos [9]	600	0.4	1.5	± 5	0.5	Low	~\$1.5k
BCN3D Moveo [10]	625	0.3	1	± 0.1	0.5	Low	~\$1k
Mirobot [30]	400	0.2	0.5	± 0.2	0.5	High	~\$1.6k
Blue [11]	840	2.1	—	± 3.7	2	Medium	~\$5k
This Work (IRIS)	940	3.3	15	± 1	1.5	High	~\$0.95k

■ Best value per metric across all systems (maximum for reach, speed, acceleration, and payload; minimum for repeatability and cost). ■ Best-performing 3D-printed open-source robot arm. ■ Best-performing commercial cinema robot. ■ Best overall balance of cinematic performance, usability, and cost.

from demonstrations rather than manually specifying them [22], [23]. Reinforcement learning (RL) further generalizes through end-to-end policy optimization, but often requires large-scale interaction data and faces significant sim-to-real challenges in perception-driven control [24], [25].

Building on these insights, IRIS uses a goal-conditioned ACT formulation integrated with task-specific hardware and trained with camera expert demonstrations. By framing camera motion as a visuomotor problem, the learned policy generates obstacle-aware, goal-directed trajectories directly from RGB observations [26], eliminating the need for pre-mapped environments or hand-crafted motion costs.

III. MECHATRONICS DESIGN

IRIS is a vertically integrated cinema robot comprising a task-specific 6-DOF manipulator with brushless-DC (BLDC) actuators, joint-space impedance control, and an end-effector-mounted Intel RealSense camera D435 [31]. As shown in Figure 3, the hardware is co-designed with the task objectives to prioritize long reachability, high repeatability, and high accuracy while remaining low-cost, compact, and portable.

A. Mechanical Structure

The mechanical design of IRIS prioritizes low distal inertia to improve dynamic responsiveness and achieve smooth precise camera motion. The arm provides approximately 1 m of reach, supports up to 1.5 kg payload, and has a total system mass of approximately 8.5 kg. The complete hardware system costs under \$1,000 USD in materials, including the embedded computer and the camera unit. To reduce reflected inertia at the end effector, actuators are designed to be concentrated near the base. The elbow pitch joint employs a HTD-5M timing belt transmission, enabling proximal motor placement. The wrist design is achieved via a two-motor differential, driving pitch and roll actuation. This design eliminates wrist-mounted motors, reducing total distal inertia at the wrist. The mechanical structure explicitly decouples translational and rotational DOF, reducing kinematic coupling and allowing

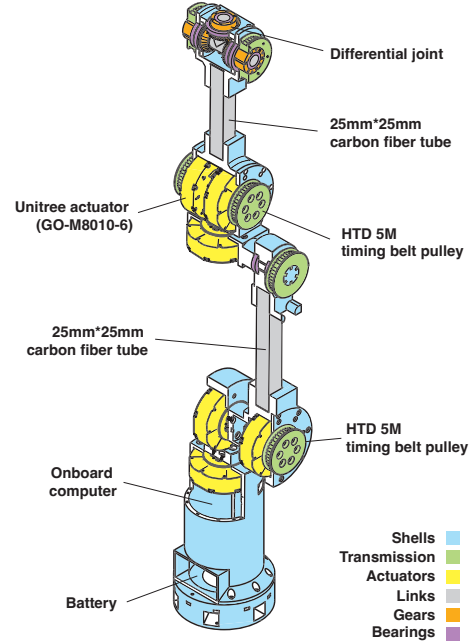


Fig. 3. IRIS hardware overview: lightweight task-specific architecture with relocated actuation and a differential wrist.

faster and more stable inverse kinematics (IK) solutions. The robot links consist of two 20×20 mm carbon-fiber tube links and 24 custom 3D-printed components, including joint housings, timing-belt pulleys, and structural supports. All non-standard structural components are fabricated using 3D-printing, enabling accessible, low-cost reproduction without specialized machining.

B. Electronics

The system contains six actuators daisy-chained over a half-duplex RS-485 bus, interfaced to the host computer via a USB-to-serial adapter. While IRIS supports embedded operation on an NVIDIA Jetson Nano, experiments were conducted on a laptop with an NVIDIA RTX 4080 GPU for low-latency

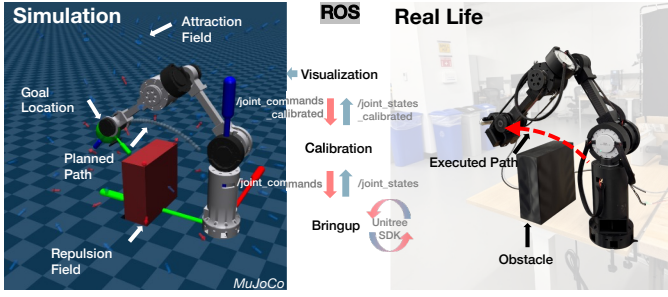


Fig. 4. Sim-to-real execution of planner-generated trajectories. A classical potential-field planner generates collision-free reference paths in simulation (left), which are then executed on the physical IRIS robot (right) via a ROS-based control stack for validation and comparison.

inference. Power is supplied by a 3S Li-Po battery (24 V nominal) or regulated DC input, with an average draw of 4 A (around 100 W) and 1 A at idle. It peaks up to 10 A during high-dynamic motion.

C. Actuation and Control Stack

To address the conflicting requirements of high-speed and high accuracy, we use Quasi-Direct Drive (QDD) for our actuation [32]. QDD offers high torque density and backdrivability while maintaining low costs (approx. \$80 USD per unit for our educational priced actuator). IRIS is actuated by Unitree GO-M8010-6 BLDC motors [33], featuring a 6.33:1 planetary reduction and integrated field-oriented control (FOC). The actuators operate at a nominal 24 V (12-30 V range), delivering peak torques of 23.7 N·m and velocities up to 30 rad/s. Each 530 g unit provides 15-bit absolute position sensing, with real-time feedback for velocity and estimated torque. Our low-level control architecture decouples high-level planning from a 200 Hz impedance control loop [34]. The pipeline streams joint positions, velocities, torque targets, and impedance gains through the vendor software development kit (SDK), enabling compliant yet precise motion execution. To ensure trajectory smoothness, commands are pre-processed through a first-order low-pass filter ($\alpha = 0.08$) and velocity-limited ramping (0.6-1.0 rad/s). For safety, a 250 ms command timeout triggers an automatic transition to a passive state estimation mode.

IV. SYSTEM INTEGRATION

A. MuJoCo Simulation

A high-fidelity MuJoCo simulation of IRIS is developed for safe policy evaluation and trajectory visualization (Figure 4). The MuJoCo model captures IRIS’s accurate kinematics, inertial properties, and collision geometry to support motion planning validation. It also enables systematic evaluation of classical path-planning baselines, including sampling-based rapidly-exploring random tree star (RRT*), prior to real-world deployment. To minimize the sim-to-real gap, we adopt similar actuator modeling protocols similar to the Unitree GO2 [35], Disney Research’s original bipedal robot [36] and their recent Olaf [37], which all use the same actuator as IRIS. Joint parameters (damping, armature inertia, friction) are tuned to

match empirical step responses from the physical system, resulting in reduced sim-to-real gap.

B. ROS Interface and Calibration

We develop a custom ROS package for IRIS that bridges the Unitree SDK, MuJoCo, and the learning stack, as seen in Figure 4. A hardware bringup node manages low-level actuator I/O through the `/joint_states` and `/joint_commands` topics. Since the actuators use incremental encoders, a startup homing sequence is required to initialize IRIS by placing it in a vertical upright pose to establish absolute joint references. Another calibration node maps raw encoder readings to kinematic coordinates (`_calibrated`) using the homing offsets and does the differential wrist transformation. The 200 Hz bringup node synchronizes real-time state feedback with the MuJoCo model and applies velocity-limited commands, enabling reproducible sim-to-real deployment.

C. Inverse Kinematics

We use a numerical Jacobian-based IK solver to handle redundancy and singularities without requiring an analytical formulation. At each control step, the Cartesian pose error is $e = [p_t - p, r_t - r]$, where $p, r \in \mathbb{R}^3$ are the current end-effector position and orientation, respectively, and p_t, r_t are the corresponding targets. Joint updates are computed via damped least squares, $\Delta q = J^\top (JJ^\top + \lambda I)^{-1} e$, where $J \in \mathbb{R}^{6 \times n}$ is the Jacobian, n the number of joints, and $\lambda > 0$ the damping coefficient. An intermediate joint estimate is computed as $q_{ik} = q + \eta \Delta q$, where $q \in \mathbb{R}^n$ is the current joint state and η is a step size. The command is smoothed using an exponential moving average, $q_{cmd,t} = \alpha q_{ik,t} + (1 - \alpha) q_{cmd,t-1}$, where $\alpha \in (0, 1)$ is the smoothing coefficient and $q_{cmd,t}$ denotes the final commanded joint configuration at time t . The filtered command q_{cmd} is applied to the controller and used to initialize the next IK iteration.

V. IMITATION LEARNING FOR MOTION CONTROL

To enable intuitive and autonomous cinematic control, we introduce a learning-based framework where users specify shots via a single target image rather than explicit geometric waypoints. Our design uses a goal-conditioned adaptation of ACT [19], augmented with a conditional variational autoencoder (CVAE). This architecture explicitly models the multimodal distribution of expert cinematic styles, allowing for diverse valid trajectories. By formulating camera motion as an IL problem, the learning pipeline is able to generate human-like, obstacle-aware trajectories that converge to the desired framing. Figure 5 provides an overview of the policy architecture.

A. Problem Formulation

The camera motion control problem is formulated as a goal-conditioned partially observable Markov decision process (POMDP). At time step t , the observation is defined as

$$o_t = (I_t, q_t), \quad (1)$$

where $I_t \in \mathbb{R}^{H \times W \times 3}$ denotes the RGB image captured by the end-effector-mounted camera at time t , and $q_t \in \mathbb{R}^6$ represents

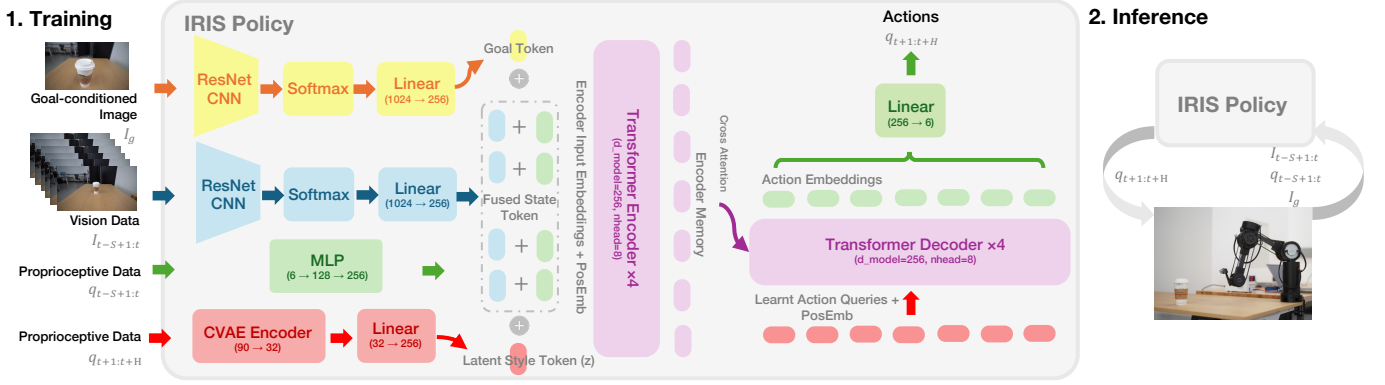


Fig. 5. IRIS policy architecture. During training (left), the model conditions on observation history, a goal image, and a CVAE-encoded latent style token z derived from the ground-truth future trajectory. Visual inputs pass through a shared ResNet-18 and Spatial Softmax to preserve spatial coordinates, then fuse with proprioception into temporal tokens. A transformer decoder predicts the 15-step joint trajectory $\hat{q}_{t+1:t+H}$. At inference (right), the CVAE branch is replaced with $z = 0$ for deterministic execution.

the robot joint configuration. The task goal is specified by a *goal image* $I_g \in \mathbb{R}^{H \times W \times 3}$ that encodes the desired visual framing. To capture the multimodality of feasible cinematic motions, we introduce a latent variable $z \in \mathbb{R}^{d_z}$ and model the policy as a conditional variational distribution

$$\hat{q}_{t+1:t+H} = \pi_\theta(o_{t-S+1:t}, I_g, z), \quad (2)$$

where π_θ is a parameterized policy with learnable parameters θ , $o_{t-S+1:t} = \{o_{t-S+1}, \dots, o_t\}$ denotes a history of S observations, and $\hat{q}_{t+1:t+H} = \{\hat{q}_{t+1}, \dots, \hat{q}_{t+H}\}$ is the predicted sequence of absolute joint configurations over a horizon of H time steps.

B. Policy Architecture

We parameterize π_θ using a DETR-style ACT [19] augmented with a CVAE to capture diverse cinematic styles. A frozen ResNet-18 backbone encodes the observation history and goal image into spatial feature maps, which are processed via spatial softmax to explicitly preserve feature coordinates. These visual features are fused with proprioceptive embeddings at each timestep to form *fused state tokens*. During training, a CVAE encoder maps the ground-truth trajectory to a latent variable z , which is projected into a *latent style token* and prepended to the transformer input as [Style, State, Goal]. At deployment, z is replaced with a zero vector for deterministic execution. The transformer encoder processes this multimodal context, enabling the decoder to predict future joint trajectories $\hat{q}_{t+1:t+H}$ via cross-attention. The complete architecture comprises 19.1M parameters, with 11.2M in the frozen backbone and a 7.9M trainable policy, of which 93% (7.4M) are in the transformer layers. The remaining parameters are distributed between the input layers (404k) and the training-only CVAE encoder (105k).

C. Loss Function

The policy is trained by minimizing a weighted composite objective $\mathcal{L}$, which is given by

$$\mathcal{L} = \mathcal{L}_{\text{mse}} + \mathcal{L}_{\text{kl}} + \mathcal{L}_{\text{smooth}}. \quad (3)$$

Algorithm 1 IRIS ACT-CVAE Policy Training

Require: Dataset $\mathcal{D}$, history length S , horizon H

- 1: Initialize policy parameters θ and CVAE encoder ϕ
- 2: Initialize optimizer (AdamW)
- 3: **for** each training iteration **do**
- 4: Sample $(I_{t-S+1:t}, q_{t-S+1:t}, I_g, q_{t+1:t+H}^*) \sim \mathcal{D}$
- 5: Sample latent $z \sim q_\phi(z | q_{t+1:t+H}^*)$
- 6: $\hat{q}_{t+1:t+H} \leftarrow \pi_\theta(I_{t-S+1:t}, q_{t-S+1:t}, I_g, z)$
- 7: Update (θ, ϕ) using loss $\mathcal{L}$ from Eq. (3)
- 8: **end for**

Algorithm 2 IRIS ACT-CVAE Policy Deployment

Require: Trained policy π_θ , goal image I_g

- 1: Initialize observation buffer of length S
- 2: Initialize ROS I/O
- 3: **while** ROS is running **do**
- 4: Receive (I_t, q_t) and append to buffer
- 5: **if** buffer not full **then**
- 6: **continue**
- 7: **end if**
- 8: $\hat{q}_{t+1:t+H} \leftarrow \pi_\theta(I_{t-S+1:t}, q_{t-S+1:t}, I_g, z=0)$
- 9: $q^{\text{cmd}} \leftarrow \text{EMA}(\hat{q}_{t+1})$
- 10: Publish q^{cmd} at control rate f_c
- 11: **end while**

In this objective, $\mathcal{L}_{\text{mse}}$ is defined as

$$\mathcal{L}_{\text{mse}} = \frac{1}{H} \sum_{i=1}^H \|\hat{q}_{t+i} - q_{t+i}^*\|_2^2, \quad (4)$$

enforcing accurate trajectory tracking in joint space via reconstruction loss, $\mathcal{L}_{\text{kl}} = \beta D_{\text{KL}}(q_\phi(z | \cdot) \| \mathcal{N}(0, I))$ regularizes the variational latent distribution using KL divergence, and

$$\mathcal{L}_{\text{smooth}} = \lambda \sum_{i=2}^H \|\hat{q}_{t+i} - \hat{q}_{t+i-1}\|_1, \quad (5)$$

penalizes high-frequency temporal variations in the predicted joint configurations to ensure smoothness.

D. Data Collection

Since cinematic nuances are difficult to model mathematically, we use camera experts to teach directly on the physical

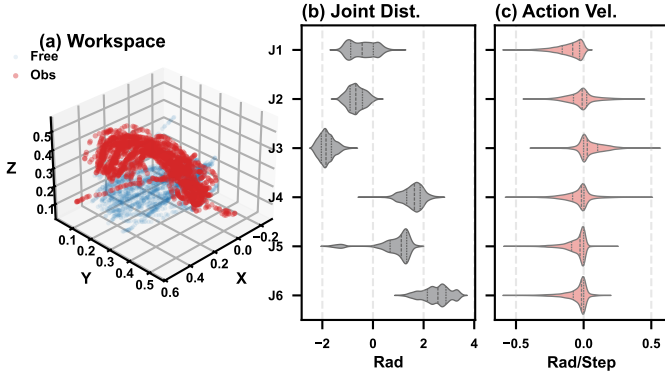


Fig. 6. Demonstration dataset coverage. (a) End-effector workspace coverage over all demonstrations. (b) Joint-space distributions of the six joints. (c) Per-step joint displacement magnitude, indicating bounded control actions.

IRIS hardware. An operator manually guides the end-effector to produce a push-in shot tracking a coffee cup in the zero-torque actuation mode. We collect synchronized demonstration data via ROS, recording joint states at 200 Hz and RGB images at 30 Hz. Demonstrations are segmented into complete shot episodes and converted into training clips using a sliding window with observation horizon $S=8$ and prediction horizon $H=15$, where the final frame of each episode serves as the goal image. The dataset consists of 132 episodes and 13,954 clips, balanced between unobstructed (6,680 clips) and obstacle-avoidance (7,274 clips) push-in shots. Episodes are split at the episode level into train/validation/test sets (80/10/10), yielding 11,039 training, 1,306 validation, and 1,609 test clips. Figure 6 visualizes the dataset distribution and expert trajectories.

E. Training

All imitation learning policies are trained with batch size 64 on a single NVIDIA RTX 4090 GPU (Alienware X16, Intel i9, Ubuntu 22). The complete hyperparameters used during training can be seen in Table II. Models are trained on 11,039 training clips and validated on 1,306 validation clips. All RGB inputs are resized to 224×224 and normalized using ImageNet statistics before the training loop to reduce the CPU bottleneck speed down. This pre-processing step reduces per-epoch training time from over 20 minutes to under 5 minutes. Each epoch consists of 345 gradient update steps, resulting in approximately 34,500 total updates over 100 epochs. On the RTX4090, one epoch takes about 300 seconds, resulting in a total training time of roughly 8 hours.

F. Policy Deployment

We deploy the policy on an ROS-based controller running at 10 Hz, using a synchronizer to align live RGB images and joint states into a history buffer of length $S = 8$. At each control step, the model predicts a trajectory of future absolute joint positions $\hat{q}_{t:t+H}$. We employ a receding-horizon strategy with a lookahead of $k = 1$ to compensate for system latency. Before actuating, the raw target $\hat{q}_{t+k}$ is first clamped to a maximum deviation of $\delta_{\max} = 0.2$ rad from the current physical state. Subsequently, an exponential moving average (EMA) filter with

TABLE II
KEY MODEL AND TRAINING HYPERPARAMETERS.

Parameter	Value
Observation horizon S	8
Prediction horizon H	15
Visual backbone	ResNet-18 (ImageNet-pretrained)
Image resolution	224×224
Transformer dimension d	256
Attention heads	8
Encoder / decoder layers	4 / 4
Latent dimension z	32
Batch size	64
Optimizer	AdamW
Learning rate	1×10^{-4}
Training epochs	100
KL weight β	0.01
Smoothness weight λ_{smooth}	0.01

$\alpha = 0.3$ is applied to suppress high-frequency jitter, calculated as $q_{t+1}^{\text{cmd}} = \alpha \hat{q}_{t+k} + (1 - \alpha) q_t^{\text{cmd}}$. Inference is performed on a single NVIDIA RTX 3070 with a forward-pass latency less than 10 ms.

VI. EXPERIMENTS

The evaluation comprises two components: a low-level control study to assess the intrinsic motion accuracy of IRIS, and a high-level learning-based motion control evaluation to validate the effectiveness of goal-conditioned shot execution. All experiments are conducted on the physical IRIS robot under identical workspace layouts and configurations.

A. Low-Level Control Evaluation

We first validate the robot’s hardware fidelity to ensure it supports learning-based control; see Figure 7.

Repeatability. We repeatedly execute a fixed joint-space trajectory for $K = 10$ trials, stopping at four distinct 3D waypoints, and evaluate repeatability at each point as the mean deviation from the centroid. We define the repeatability as

$$E_{\text{rep}} = \frac{1}{K} \sum \|\mathbf{x}_T^k - \bar{\mathbf{x}}_T\|_2. \quad (6)$$

Tracking Accuracy. Path-tracking accuracy is evaluated by executing a predefined circular trajectory in 3D space for two cycles and comparing the desired end-effector trajectory with the executed trajectory reconstructed from joint encoder measurements. We formulate the tracking error using the root mean square error (RMSE)

$$E_{\text{track}} = \frac{1}{T} \int \|\mathbf{x}(t) - \mathbf{x}^*(t)\|_2^2 dt. \quad (7)$$

B. Shot Automation Experiment Setup

We evaluate the learning-based policy’s style reproduction and generalization on two tasks: (i) *Push-in (Unobstructed)*, a dolly-in toward a target (coffee cup) guided by a goal image; and (ii) *Push-in (Obstacle Avoidance)*, which adds a cubic obstacle ($10 \times 25 \times 25$ cm) to the path. Each trial begins from a different initial pose and is repeated 10 times to assess shot quality and success rate. The experiment setup can be referred to Figure 8 on the left.

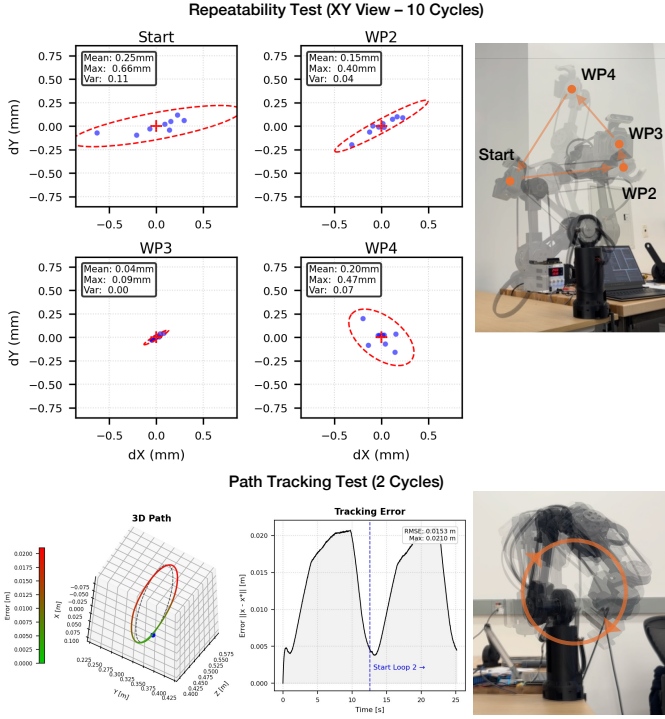


Fig. 7. Low-level evaluation. Top: End-effector XY scatter over $K = 10$ trials at start and WP2-WP4. Bottom: Reference vs. executed 3D trajectories and Cartesian error. IRIS achieves sub-millimeter repeatability and centimeter-scale accuracy.

C. Benchmarks

We compare our approach against three state-of-the-art motion control strategies. Namely, (i) *Human Expert Replay*, which utilizes a direct "Teach and Repeat" replay of expert demonstrations; (ii) *Classical Planner (RRT*)*, a geometric baseline generating collision-free open-loop trajectories by implementing joint-space RRT* [13] in MuJoCo with capsule-based collision checking (7.5 cm safety radius) for offline planning between IK-solved configurations; and (iii) *Vanilla BC*, a deterministic vanilla BC baseline [38] that directly regresses future absolute joint positions from RGB observations, goal images, and joint history using a Multi-Layer Perceptron (MLP) and ResNet backbone for feature extraction.

D. Evaluation Metrics

The system is evaluated on six core metrics that quantify cinematic quality and control precision.

- *Visual Alignment ($\mathcal{S}_{vis}$)*. Quantifies semantic similarity between the last (I_{last}) and goal (I_g) images using feature embeddings $\phi(\cdot)$ [39] from the penultimate layer of a pre-trained ResNet-18 [40], [41].

$$\mathcal{S}_{vis} = \frac{\phi(I_{last}) \cdot \phi(I_g)}{\|\phi(I_{last})\|_2 \|\phi(I_g)\|_2} \quad (8)$$

- *Success Rate*. The percentage of trials where the robot reaches the target configuration without collision and achieves a visual alignment score $\mathcal{S}_{vis} > 0.85$ (aligned).

- *Cartesian Smoothness ($\mathcal{J}_{cart}$)*. Measures physical stability via the magnitude of the end-effector jerk [42]. We compute the end-effector position $\mathbf{x}_t \in \mathbb{R}^3$ from joint states using a MuJoCo-based forward kinematics solver and approximate the third-order time derivative:

$$\mathcal{J}_{cart} = \|\ddot{\mathbf{x}}_t\|_2 \approx \left\| \frac{\Delta^3 \mathbf{x}_t}{(\Delta t)^3} \right\|_2 \quad (9)$$

- *Framing Error ($\mathcal{E}_{frame}$)*. The Euclidean pixel distance between last image center $\mathbf{c}$ and the target object's centroid $\mathbf{p}$, detected in real-time via YOLOv8 (Nano) [40]:

$$\mathcal{E}_{frame} = \|\mathbf{p} - \mathbf{c}\|_2 \quad (10)$$

- *Subject Retention Rate (SRR)*. The percentage of the trajectory where the target object remains within the central 50% of the camera frame [26], ensuring consistent subject tracking.
- *Inference Latency*. The average wall-clock time required for the policy to process visual observations and output motor commands.

E. Ablation Studies

We validate our architecture design through two ablation studies, with 10 trials for each policy.

Action Space Formulation: We compare our proposed *absolute* control strategy ($q_{t+1}^{cmd} = \hat{q}_t$) against an *incremental* action output ($q_{t+1}^{cmd} = q_t^{curr} + \Delta \hat{q}_t$) to verify if direct absolute position regression yields better global consistency for cinematic trajectories.

Input Modality: We analyze input modalities importance by comparing the full configuration ($\mathcal{O} = \{I_{t-S:t}, q_{t-S:t}, I_g\}$) against two variants: (i) *Visual-Only* ($\mathcal{O} = \{I_{t-S:t}, I_g\}$), removing proprioception; and (ii) *RGB-Only* ($\mathcal{O} = \{I_{t-S:t}\}$), removing both goal conditioning and proprioception to test implicit task inference. The ablation studies results can be found in Table III.

F. Results and Discussion

The low-level control results validate the hardware integrity of IRIS and the effectiveness of our low-level control architecture in maintaining the precision required for cinematic tasks. Evaluation of the system's mechanical consistency shows that IRIS's repeatability remains sub-millimeter across all measured waypoints: 0.25 mm (Start), 0.15 mm (WP2), 0.04 mm (WP3), and 0.20 mm (WP4), with a maximum observed deviation of 0.66 mm. Furthermore, the joint-space impedance controller achieves a RMSE of 1.53 cm and a maximum error of 2.10 cm. This performance indicates sufficient control fidelity and stability for reliable downstream visuomotor policy deployment in complex environments.

For the high-level policy experiments, the comparison metrics are summarized in Table III. Our ACT-CVAE policy achieves a 90% success rate, significantly outperforming the classical planner (10%). This performance gap highlights the advantage of closed-loop visuomotor control. While the open-loop planner fails against unmodeled sim-to-real gap, the learned policy

TABLE III
PERFORMANCE COMPARISON AND ABLATION STUDIES. EVALUATION OF TASK METRICS ACROSS 10 RANDOMIZED TRIALS.

Method	Success $\uparrow$ (%)	Vis. Align. $\uparrow$ (S_{vis})	Cart. Jerk $\downarrow$ (m/s^3)	Framing $\downarrow$ ($\mathcal{E}_{frame}$ px)	SRR $\uparrow$ (%)	Latency $\downarrow$ (ms)
Baselines						
Human Expert Replay	90.0	0.874	3.64	105.7	67.1	–
Classical Planner (RRT*)	10.0	0.636	0.22	219.6	10.5	–
Ablation Studies						
w/ Incremental Action Space	0.0	0.636	0.83	103.1	31.9	9.1
w/ RGB Input Only	0.0	0.584	1.65	113.0	7.2	9.0
w/ RGB + Goal (No Proprio)	0.0	0.536	1.59	162.1	7.3	9.2
ACT-CVAE (Ours)	90.0	0.847	0.61	183.6	33.1	9.2

Best value per metric. Proposed method. Human Expert and Classical Planner baselines are computed offline; therefore, latency is not reported.
 $\uparrow$ indicates higher is better; $\downarrow$ indicates lower is better.

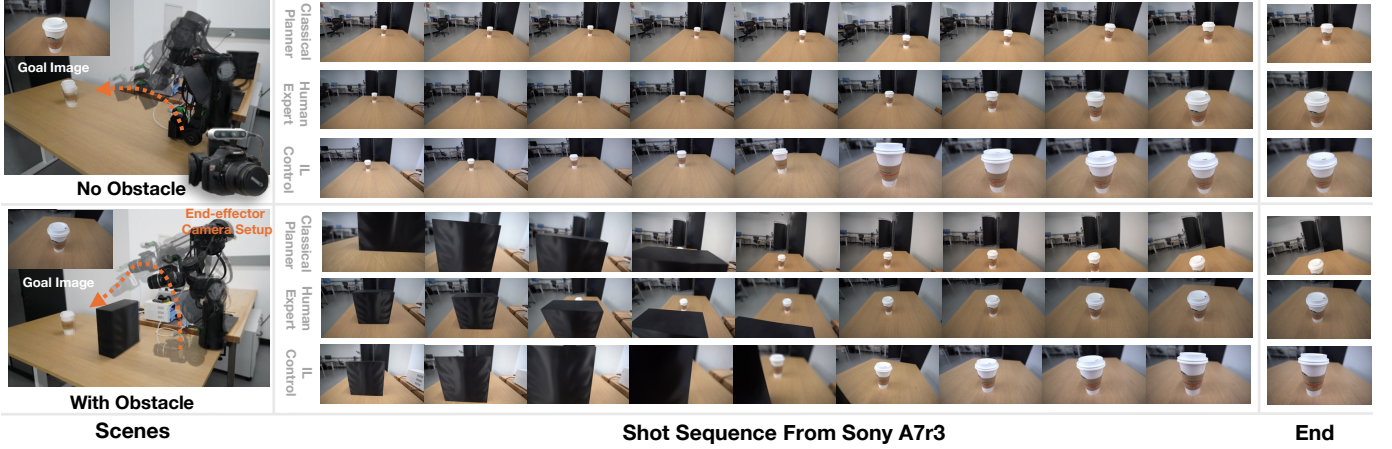


Fig. 8. Qualitative shot reproduction results. Left: Experimental setup with the goal images for unobstructed (top) and obstacle (bottom) scenes. Right: Temporal image sequences comparing the Classical Planner, Human Expert, and our IL policy. Unlike the planner, which fails against unmodeled obstacles, our policy successfully maintains the desired framing while avoiding collisions, closely matching expert demonstrations.

dynamically adapts the trajectory and corrects the final pose to match the visual goal. Additionally, our policy functions as a motion filter, producing trajectories significantly smoother than human experts ($0.61 m/s^3$ vs. $3.64 m/s^3$ jerk) by rejecting high-frequency jitter. This is achieved at the expense of responsiveness drops, with a lower SRR (33.1%) compared to human operators (67.1%). Despite this lag, it outperforms the classical planner (10.5%), showing effective semantic object awareness where geometric baselines fail. The qualitative shot results are further given in Figure 8.

Ablation studies show the importance of action space formulation, with incremental control failed completely (0% success). We suspect that this is due to integration drift exacerbated by hardware compliance, whereas predicting *absolute* joint positions provides the necessary self-correction for low-cost hardware. Furthermore, removing either proprioception or goal conditioning also resulted in total failure, verifying the necessity of full multimodal context.

Despite these capabilities, the broader robustness and functionalities of the system remain bounded by three factors: mechanical flexion during high-torque maneuvers, limited dataset diversity across scenes and objects, and a restricted

repertoire of cinematic shot types.

VII. CONCLUSIONS AND FUTURE WORK

We presented IRIS, a low-cost, fully 3D-printed cinema robot system that integrates task-specific hardware with learning-based motion generation for automated shot execution. By coupling a cinematography-oriented mechanical design with visuomotor imitation learning, IRIS achieves smooth and repeatable camera motion without reliance on expensive commercial platforms or expert-driven programming workflows. These results demonstrate that tightly designed and integrated hardware learning systems can make robotic cinematography more accessible and practical for a wider range of creators and research applications. The current design can be extended in various aspects: at the hardware side, stiffening the chassis and optimizing actuator selection can minimize flexion. On the learning side, expanding the dataset to more diverse scenes and objects will enable real-time sequencing of multiple visual goals, supporting complex long-horizon camera motions such as compound dolly, crane, and pan trajectories. Future work will explore these directions.

REFERENCES

- [1] E. P. Atherton, D. Thomasson, M. U. Conti, and H. Kerrick, "Robotic camera control via motion capture," U.S. Patent US10427305B2, Oct., 2019.
- [2] R. Fielding, *The Technique of Special Effects Cinematography*, 4th ed. Focal Press, 1985.
- [3] C. de la Cruz *et al.*, "Robots and the moving camera in cinema, television and digital media," *Digital Creativity*, vol. 27, no. 3-4, pp. 193–205, 2016.
- [4] Mark Roberts Motion Control. (2024) Faq: What is motion control? Accessed: 2026-02-01. [Online]. Available: <https://www.mrmoco.com/faq/>
- [5] Motorized Precision, "Kira Cinema Robot," <https://www.motorizedprecision.com/kira>, 2026, accessed: 2026-02-01.
- [6] S. Mick, M. Lapeyre, P. Rouanet, C. Halgand, J. Benois-Pineau, F. Paclet, D. Cattaert, P.-Y. Oudeyer, and A. de Rugy, "Reachy, a 3D-printed human-like robotic arm as a testbed for human-robot control strategies," *Frontiers in Neurobotics*, vol. 13, p. 65, 2019.
- [7] OpenArm, "Openarm: A fully open-source humanoid robot arm for physical ai research," <https://openarm.dev>, 2025, accessed: Jan. 21, 2026.
- [8] S. Balasubramanian, A. Melendez-Calderon, A. Roby-Brami, and E. Burdet, "On the analysis of movement smoothness," *Journal of neuroengineering and rehabilitation*, vol. 12, no. 1, pp. 1–11, 2015.
- [9] Arctos Robotics, "Arctos-a 3d printed, open source robotic arm," 2020, accessed: 2024. [Online]. Available: <https://github.com/SkyeSon/ARCTOS>
- [10] BCN3D Technologies, "Bcn3d moveo: A fully open source 3d printed robot arm," 2016, accessed: 2024. [Online]. Available: <https://www.bcn3d.com/bcn3d-moveo-open-source-robot/>
- [11] D. V. Gealy, S. McKinley, B. Yi, P. Wu, P. R. Downey, G. Balke, A. Zhao, M. Guo, R. Thomasson, A. Sinclair, P. Cuellar, Z. McCarthy, and P. Abbeel, "Quasi-direct drive for low-cost compliant robotic manipulation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 437–443.
- [12] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [13] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.
- [14] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 489–494.
- [15] J. Schulman, J. Ho, A. Lee, I. Awwal, H. Bradlow, and P. Abbeel, "Finding locally optimal, collision-free trajectories with sequential convex optimization," in *Robotics: Science and Systems*, vol. 9, 2013, pp. 1–10.
- [16] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *J. Mach. Learn. Res.*, vol. 17, no. 1, p. 1334–1373, Jan. 2016.
- [17] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, vol. 15. PMLR, 11–13 Apr 2011, pp. 627–635.
- [18] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Proceedings of the 30th International Conference on Neural Information Processing Systems*, ser. NIPS'16, 2016, p. 4572–4580.
- [19] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems (RSS)*, 2023.
- [20] K. Bekris, K. Hauser, S. Herbert, J. Yu, C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *Int. J. Rob. Res.*, vol. 44, no. 10–11, p. 1684–1704, Sep. 2025.
- [21] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 17–23 Jul 2022, pp. 9902–9915.
- [22] B. D. Ziebart, A. Maas, J. A. Bagnell, and A. K. Dey, "Maximum entropy inverse reinforcement learning," in *Proceedings of the 23rd National Conference on Artificial Intelligence - Volume 3*, ser. AAAI'08. AAAI Press, 2008, p. 1433–1438.
- [23] C. Finn, S. Levine, and P. Abbeel, "Guided cost learning: Deep inverse optimal control via policy optimization," in *Proceedings of The 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 48, 20–22 Jun 2016, pp. 49–58.
- [24] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 1861–1870.
- [25] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013. [Online]. Available: <https://doi.org/10.1177/0278364913495721>
- [26] F. Chaumette and S. Hutchinson, "Visual servo control. i. basic approaches," *IEEE Robotics & Automation Magazine*, vol. 13, no. 4, pp. 82–90, 2006.
- [27] Motorized Precision, "Colossus: The Largest & Most Powerful Cinema Motion Control Robot," <https://www.motorizedprecision.com/colossus>, 2021, accessed: 2026-02-02.
- [28] LBBOnline. (2023) A guide to motion control filmmaking. Accessed: 2026-02-01. [Online]. Available: <https://lbbonline.com/news/a-guide-to-motion-control-filmmaking>
- [29] Haddington Dynamics, "Dexter: Open source 5-axis robot arm," 2016, accessed: 2024. [Online]. Available: <https://hackaday.io/project/158779-dexter>
- [30] "Wlkata — robotics and stem education equipment," <https://www.wlkata.com>, 2026, official website, accessed on 2026-02-02.
- [31] Intel Corporation, *Intel® RealSense™ Depth Camera D400-Series Datasheet*, Intel Corporation, March 2023, document Number: 337029-013.
- [32] D. V. Gealy, S. McKinley, B. Yi, P. Wu, P. R. Downey, G. Balke, A. Zhao, M. Guo, R. Thomasson, A. Sinclair, P. Cuellar, Z. McCarthy, and P. Abbeel, "Quasi-direct drive for low-cost compliant robotic manipulation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 437–443.
- [33] Unitree Robotics, *Unitree Go-M8010-6 Actuator Technical Specifications*, Hangzhou Yushu Technology Co., Ltd., 2024, high-performance robot joint motor with integrated planetary reducer and FOC driver. [Online]. Available: <https://www.unitree.com/go1/motor>
- [34] P. M. Wensing, A. Wang, S. Seok, D. Otten, J. H. Lang, and S. Kim, "Proprioceptive actuator design for dynamic robots," *IEEE Transactions on Robotics*, vol. 33, no. 6, pp. 1450–1464, 2018.
- [35] Unitree Robotics. (2024) Unitree go2 quadruped robot. Accessed: 2026-02-01. [Online]. Available: <https://www.unitree.com/go2>
- [36] R. Grandia, E. Knoop, M. A. Hopkins, G. Wiedebach, J. Bishop, S. Pickles, D. Müller, and M. Bächer, "Design and control of a bipedal robotic character," in *Robotics: Science and Systems (RSS)*, 2024.
- [37] D. Müller, E. Knoop, D. Mylonopoulos, A. Serifi, M. A. Hopkins, R. Grandia, and M. Bächer, "Olaf: Bringing an animated character to life in the physical world," 2025.
- [38] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016.
- [39] J. Bromley, J. Bentz, L. Bottou, I. Guyon, Y. LeCun, C. Moore, E. Säckinger, and R. Shah, "Signature verification using a 'siamese' time delay neural network," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 7, no. 4, pp. 669–688, 1993.
- [40] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [41] D. Reis, J. Kupec, J. Hong, and A. Daoudi, "Real-time flying object detection with yolov8," 2024.
- [42] N. Hogan, "An organizing principle for a class of voluntary movements," *Journal of Neuroscience*, vol. 4, no. 11, pp. 2745–2754, 1984.